

SOC/L1 - 7.4. Man-in-the-Middle Detection

Task 1 - Introduction

Man-in-the-middle (MITM) attacks represent one of the most insidious threats in network security. In these attacks, attackers position themselves between legitimate communication endpoints to intercept, modify, or redirect traffic. From a blue team perspective, detecting these attacks requires a multi-layered approach combining network monitoring, certificate validation, and behavioral analysis.

Task 2 - Lab Connection

A routine network monitoring alert at **Acme Corp** revealed unusual traffic patterns suggesting a possible **Man-in-the-Middle (MITM)** attack inside the corporate LAN. Over several days, an attacker quietly intercepted communications, redirected connections, and captured user credentials.

Three chained MITM techniques.

- **ARP Spoofing** (network interception).
- **DNS Spoofing** (redirection).
- **SSL Stripping** (credential capture).

Task 3 - MITM Attacks

A Man-in-the-Middle (MITM) attack is a cyberattack where an attacker secretly intercepts and potentially alters communication between two parties, such as a user and a service, without their knowledge. The attacker may eavesdrop to steal sensitive data like credentials and credit card info or inject malicious content. These attacks can target any organization or individual, especially where encryption or authentication is weak.

How MITM Attacks Work

MITM attacks generally involve two main steps:

- **Interception:** The attacker inserts themselves into a communication stream, often by exploiting weaknesses in network protocols or by using techniques like ARP, DNS or IP spoofing.
- **Manipulation/Decryption:** The attacker tries to access or modify the communication, decrypting encoded data or injecting harmful content, such as altered website responses or fake login forms.

Common Types of MITM Attacks

- Packet sniffing: Capturing unencrypted data packets exchanged over a network, often on open Wi-Fi.
- Session hijacking: Stealing and using session tokens to impersonate users.
- SSL stripping: Downgrading HTTPS connections to insecure HTTP to steal or alter data transferred.
- DNS spoofing: Redirecting legitimate website traffic to fraudulent domains by manipulating DNS responses.
- IP spoofing: Crafting malicious IP packets that appear to come from trusted systems.
- Rogue Wi-Fi access point: Creating fake networks to intercept user traffic.

Situating MITM within the Cyber Kill Chain

Man-in-the-Middle is a versatile technique that adversaries primarily leverage during the Exploitation and Installation phases.

- **As an Exploitation Technique:** A MITM attack manipulates core protocols like ARP and DNS to intercept network channels, breaching network integrity and establishing an initial foothold for eavesdropping or manipulation.
- **As an Installation Vector:** Once in control of the data stream, the attacker can inject malware (like browser exploits or RATs) into unencrypted downloads to establish persistence on the victim's system.

Task 4 - Detecting ARP Spoofing

ARP Protocol (Address Resolution Protocol): It translates a known Layer 3 IP address into a physical Layer 2 MAC address so devices on the same local network can communicate with each other.

ARP Spoofing: It is a Man-in-the-Middle (MitM) technique where an attacker sends fake ARP messages across a local area network (LAN) to trick devices into linking the attacker's MAC address with a legitimate IP address on the network. ARP has **no authentication**.

Indicators of the Attack

- **Duplicate MAC-to-IP Mappings:** Multiple MAC addresses claiming the same IP address. Indicates impersonation.
- **Unsolicited ARP Replies:** High number of ARP replies without matching requests ("gratuitous ARP").
- **Abnormal ARP Traffic Volume:** A Large number of ARP packets in short intervals.
- **Unusual Traffic Routing:** Traffic rerouted through the attacker's MAC.
- **Gateway Redirection Patterns:** Multiple destination MACs for the same gateway IP.
- **ARP Probe / Reply Loops:** Many ARP requests with `Who has 192.168.1.x? Tell 192.168.1.y` patterns.

Network Information

Role	IP	MAC	Notes
Gateway	192.168.10.1	-	Legit router
Attacker	-	-	-
Victim	-	-	-
Domain	corp-login.acme-corp.local	-	

Network Traffic Analysis

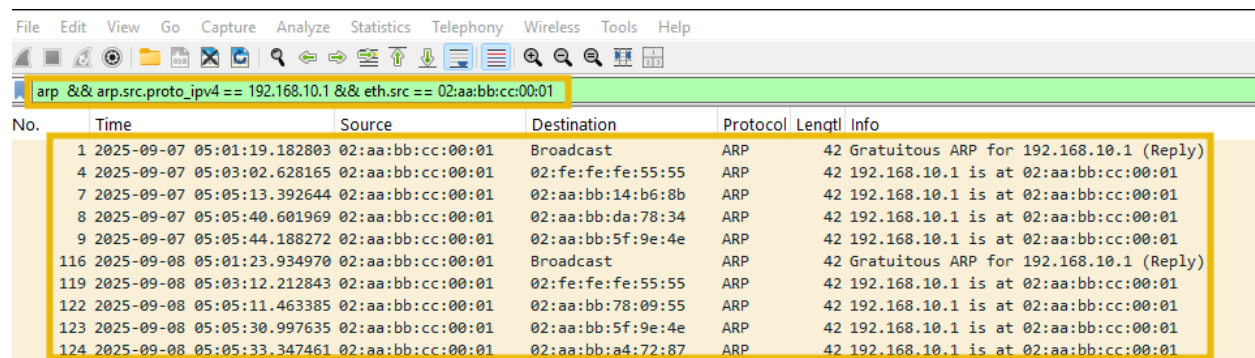
`arp.opcode == 1` (This shows all the ARP requests captured from different hosts. Asking "Who has this IP address?")

`arp.opcode == 2` (This filter displays ARP responses from various hosts. Answering "I have that IP address, and here is my MAC address.")

`arp.isgratuitous` (Gratuitous ARP Responses, A suspicious host sends many unsolicited (gratuitous) ARP replies, especially to multiple destinations. Repeated

gratuitous ARPs can indicate an attacker maintaining their poison state.)

`arp && arp.src.proto_ipv4 == 192.168.10.1 && eth.src == 02:aa:bb:cc:00:01` (This filter isolates **ARP packets sent by the specific IP address 192.168.10.1 matching the precise MAC address 02:aa:bb:cc:00:01** , usually used to verify legitimate router traffic or spot spoofed impersonators.)



The image shows a Wireshark packet capture window with a filter applied: `arp && arp.src.proto_ipv4 == 192.168.10.1 && eth.src == 02:aa:bb:cc:00:01`. The packet list shows several ARP packets, including gratuitous ARPs and requests/replies. The packet details pane shows the selected packet's structure, including Ethernet II, Internet Protocol Version 4, and ARP.

No.	Time	Source	Destination	Protocol	Length	Info
1	2025-09-07 05:01:19.182803	02:aa:bb:cc:00:01	Broadcast	ARP	42	Gratuitous ARP for 192.168.10.1 (Reply)
4	2025-09-07 05:03:02.628165	02:aa:bb:cc:00:01	02:fe:fe:fe:55:55	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01
7	2025-09-07 05:05:13.392644	02:aa:bb:cc:00:01	02:aa:bb:14:b6:8b	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01
8	2025-09-07 05:05:40.601969	02:aa:bb:cc:00:01	02:aa:bb:da:78:34	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01
9	2025-09-07 05:05:44.188272	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01
116	2025-09-08 05:01:23.934970	02:aa:bb:cc:00:01	Broadcast	ARP	42	Gratuitous ARP for 192.168.10.1 (Reply)
119	2025-09-08 05:03:12.212843	02:aa:bb:cc:00:01	02:fe:fe:fe:55:55	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01
122	2025-09-08 05:05:11.463385	02:aa:bb:cc:00:01	02:aa:bb:78:09:55	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01
123	2025-09-08 05:05:30.997635	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01
124	2025-09-08 05:05:33.347461	02:aa:bb:cc:00:01	02:aa:bb:a4:72:87	ARP	42	192.168.10.1 is at 02:aa:bb:cc:00:01

`[IP Address]` is located at `[MAC Address]` (is at explanation)

`arp.opcode == 2 && arp.src.proto_ipv4 == 192.168.10.1` (This filter isolates **ARP Replies sent specifically by the IP address 192.168.10.1** , confirming its MAC address to requesting network devices or updating their ARP tables.)

`arp.opcode == 2 && _ws.col.info contains "192.168.10.1 is at"` (This filter isolates **ARP Replies specifically stating that IP 192.168.10.1 is located at a given MAC address**, matching both the packet header opcode and Wireshark's Info column text.)

`_ws.col.info contains "192.168.10.1 is at"` This is filter isolates packets where Wireshark's Info column explicitly displays the text `"192.168.10.1 is at"` , quickly pinning down ARP announcements claiming ownership of that IP address.

`arp.opcode == 2 && arp.src.proto_ipv4 == 192.168.10.1` , which shows the same result previous.

`arp.opcode == 2 && arp.src.proto_ipv4 == 192.168.10.1 && eth.src == 02:fe[REDACTED]` The result clearly confirms that the attacker spoofs ARP. Examining the suspicious MAC address

`arp.duplicate-address-detected || arp.duplicate-address-frame` Check for Duplicate IP-to-MAC Mappings. This result indicates that the attacker successfully

performed ARP spoofing and positioned himself between the victim and the gateway.

Answer the questions below

How many ARP packets from the gateway MAC Address were observed? 10

`arp && arp.src.proto_ipv4 == 192.168.10.1 && eth.src == 02:aa:bb:cc:00:01`

network_traffic.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

arp && arp.src.proto_ipv4 == 192.168.10.1 && eth.src == 02:aa:bb:cc:00:01

No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	02:aa:bb:cc:00:01	Broadcast	ARP	42	Gratuitous ARP
4	103.445362	02:aa:bb:cc:00:01	02:fe:fe:fe:55:55	ARP	42	192.168.10.1 i
7	234.209841	02:aa:bb:cc:00:01	02:aa:bb:14:b6:8b	ARP	42	192.168.10.1 i
8	261.419166	02:aa:bb:cc:00:01	02:aa:bb:da:78:34	ARP	42	192.168.10.1 i
9	265.005469	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.10.1 i
116	86404.752167	02:aa:bb:cc:00:01	Broadcast	ARP	42	Gratuitous ARP
119	86513.030040	02:aa:bb:cc:00:01	02:fe:fe:fe:55:55	ARP	42	192.168.10.1 i
122	86632.280582	02:aa:bb:cc:00:01	02:aa:bb:78:09:55	ARP	42	192.168.10.1 i
123	86651.814832	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.10.1 i
124	86654.164658	02:aa:bb:cc:00:01	02:aa:bb:a4:72:87	ARP	42	192.168.10.1 i

Frame 1: 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface 0

Ethernet II, Src: 02:aa:bb:cc:00:01 (02:aa:bb:cc:00:01), Destination: Broadcast (ff:ff:ff:ff:ff:ff)

Address: Broadcast (ff:ff:ff:ff:ff:ff)

...1... = LG bit:

...1... = IG bit:

Source: 02:aa:bb:cc:00:01 (02:aa:bb:cc:00:01)

Address: 02:aa:bb:cc:00:01 (02:aa:bb:cc:00:01)

...1... = LG bit:

...0... = IG bit:

Type: ARP (0x0806)

Address Resolution Protocol (reply/gratuitous)

Source or Destination Hardware Address (eth.addr), 6 byte(s) | Packets: 3281 · Displayed: 10 (0.3%) | Profile: Default

What MAC address was used by the attacker to impersonate the gateway?

02:fe:fe:fe:55:55

`arp.opcode == 2 && arp.src.proto_ipv4 == 192.168.10.1`

network_traffic.pcap

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

arp.opcode == 2 && arp.src.proto_ipv4 == 192.168.10.1

No.	Time	Source	Destination	Protocol	Length	Info
8	261.419166	02:aa:bb:cc:00:01	02:aa:bb:da:78:34	ARP	42	192.168.1
9	265.005469	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.1
116	86404.752167	02:aa:bb:cc:00:01	Broadcast	ARP	42	Gratuitou
119	86513.030040	02:aa:bb:cc:00:01	02:fe:fe:fe:55:55	ARP	42	192.168.1
122	86632.280582	02:aa:bb:cc:00:01	02:aa:bb:78:09:55	ARP	42	192.168.1
123	86651.814832	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.1
124	86654.164658	02:aa:bb:cc:00:01	02:aa:bb:a4:72:87	ARP	42	192.168.1
257	202650.110137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
258	202650.310137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
259	202650.510137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
260	202650.710137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
261	202650.910137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1

Frame 257: 42 bytes on wire (336 bits), 42 b
 Ethernet II, Src: 02:fe:fe:fe:55:55 (02:fe:f
 Destination: 02:aa:bb:14:b6:8b (02:aa:bb:1
 Address: 02:aa:bb:14:b6:8b (02:aa:bb:14:
1. = LG bit:
0 = IG bit:
 Source: 02:fe:fe:fe:55:55 (02:fe:fe:fe:
 Address: 02:fe:fe:fe:55:55 (02:fe:fe:fe:
1. = LG bit:
0 = IG bit:
 Type: ARP (0x0806)
 Address Resolution Protocol (reply)
 [Duplicate IP address detected for 192.168.1

Source or Destination Hardw...dress (eth.addr), 6 byte(s); Packets: 3281 - Displayed: 24 (0.7%) | Profile: Default

How many Gratuitous ARP replies were observed for 192.168.10.1?

2 different mac received

arp.opcode == 2 && _ws.col.info contains "192.168.10.1 is at"

arp.opcode == 2 && _ws.col.info contains "192.168.10.1 is at"

No.	Time	Source	Destination	Protocol	Length	Info
8	261.419166	02:aa:bb:cc:00:01	02:aa:bb:da:78:34	ARP	42	192.168.1
9	265.005469	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.1
119	86513.030040	02:aa:bb:cc:00:01	02:fe:fe:fe:55:55	ARP	42	192.168.1
122	86632.280582	02:aa:bb:cc:00:01	02:aa:bb:78:09:55	ARP	42	192.168.1
123	86651.814832	02:aa:bb:cc:00:01	02:aa:bb:5f:9e:4e	ARP	42	192.168.1
124	86654.164658	02:aa:bb:cc:00:01	02:aa:bb:a4:72:87	ARP	42	192.168.1
257	202650.110137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
258	202650.310137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
259	202650.510137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
260	202650.710137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
261	202650.910137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
262	202651.110137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1

How many ARP spoofing packets were observed in total from the attacker? 14

arp.opcode == 2 && arp.src.proto_ipv4 == 192.168.10.1 && eth.src == 02:fe:fe:fe:55:55

No.	Time	Source	Destination	Protocol	Length	Info
257	202650.110137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
258	202650.310137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
259	202650.510137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
260	202650.710137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
261	202650.910137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
262	202651.110137	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
2230	1757349.3098...	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
2231	1757349.4598...	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
2232	1757349.6098...	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
2233	1757349.7598...	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
2234	1757349.9098...	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1
2235	1757350.0598...	02:fe:fe:fe:55:55	02:aa:bb:14:b6:8b	ARP	42	192.168.1

Frame 257: 42 bytes on wire (336 bits), 42 b

Ethernet II, Src: 02:fe:fe:fe:55:55 (02:fe:f

Destination: 02:aa:bb:14:b6:8b (02:aa:bb:1

Address: 02:aa:bb:14:b6:8b (02:aa:bb:14:

... ..1. = LG bit:

... ..0. = IG bit:

Source: 02:fe:fe:fe:55:55 (02:fe:fe:fe:

Address: 02:fe:fe:fe:55:55 (02:fe:fe:fe:

... ..1. = LG bit:

... ..0. = IG bit:

Type: ARP (0x0806)

Address Resolution Protocol (reply)

[Duplicate IP address detected for 192.168.1

Task 5 - Unmasking DNS Spoofing

DNS Protocol: It translates human-friendly domain names into machine-readable IP addresses.

DNS Spoofing (or DNS Cache Poisoning) is an attack where corrupted or fake DNS records are fed to a target, tricking their device into resolving a legitimate domain name to a malicious IP address controlled by the attacker.

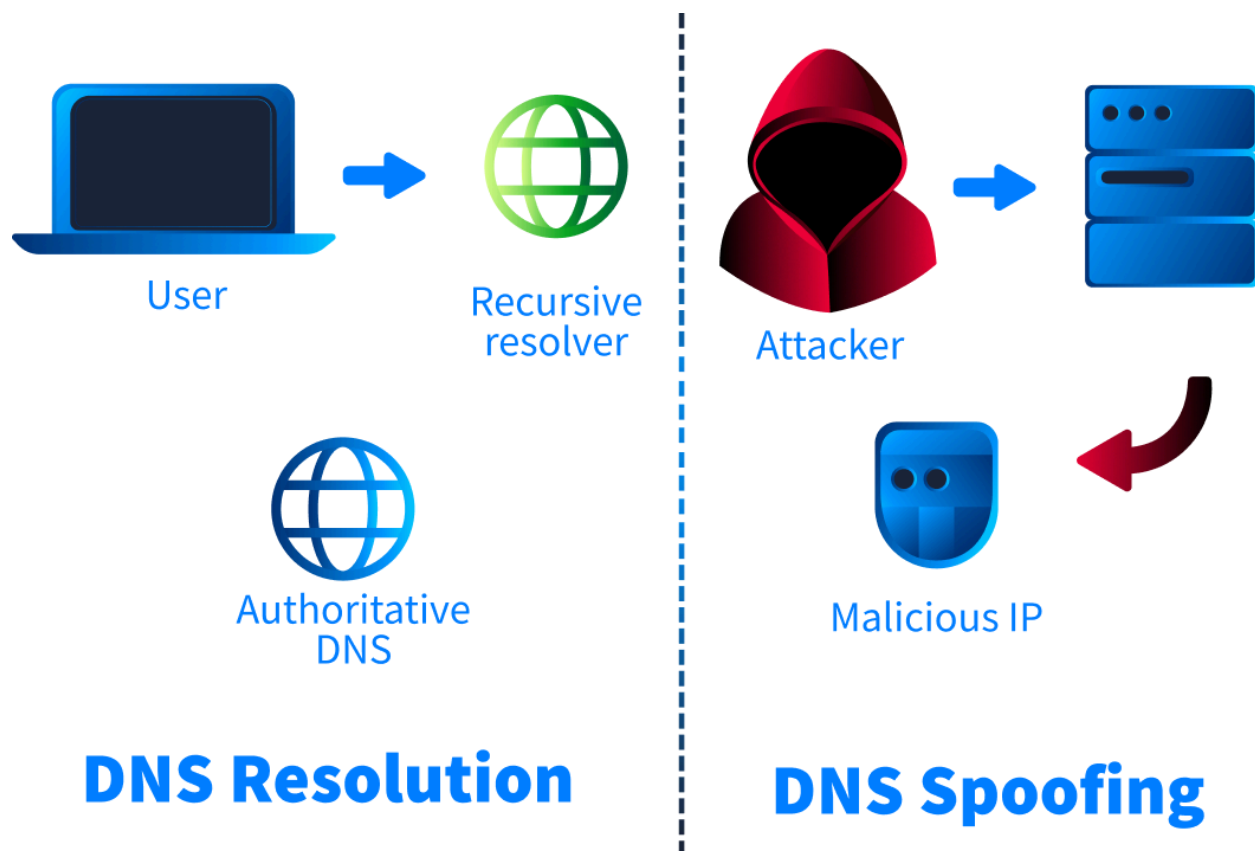
DNS Resolution is the process of translating a human-friendly domain name—like `example.com`—into a machine-routable IPv4 or IPv6 address.

- **Local Checks (Cache & Hosts File):** Before any network traffic is generated, the operating system checks its browser cache, OS DNS cache, and the local `hosts` file. If the entry exists, it skips the network entirely.
- **Recursive Resolver:** If it's a cache miss, the OS sends a UDP (or TCP) query on port 53 to the configured **Recursive Resolver**—typically provided by the

ISP or a public DNS like `1.1.1.1` or `8.8.8.8`.

- **Root Nameserver (`.`):** If the recursive resolver doesn't have the answer cached, it queries a **Root Nameserver**. The root server doesn't know the IP of `example.com`, but it knows where to find the Top-Level Domain (TLD) servers. It replies: *'I don't know, but go ask the `.com` TLD server at this address.'*
- **TLD Nameserver (`.com`):** The resolver then queries the `.com` TLD server. The TLD server responds with the IP of the **Authoritative Nameserver** delegated to manage `example.com`.
- **Authoritative Nameserver:** Finally, the resolver queries the Authoritative server. Since this server holds the actual zone files, it responds with the definitive **A** (IPv4) or **AAAA** (IPv6) record.
- **Caching & Return:** The recursive resolver passes the IP back to the client and caches it locally based on the record's **TTL (Time to Live)** value. The client's OS caches it as well, and the browser can finally initiate the TCP handshake."

The attacker is now `in the middle`, capturing everything the victim types, including their username and password.



Indicators of the Attack: Following key indicators while investigating the logs or network traffic for a potential Man-in-the-Middle attack using DNS spoofing:

- **Multiple DNS responses for the same query:** A legitimate resolver and a forged responder reply to the same query. This is the single most reliable indicator.
- **DNS response from an unexpected source:** A DNS reply arrives from an IP address **that does not match any configured resolver** (like 8.8.8.8 or your DNS server).
- **Suspiciously short TTL (Time-To-Live) values:** Attackers use very low TTLs (1 - 30s) to keep poisoned entries short-lived and reassert control.
- **Unsolicited DNS responses:** A DNS reply appears without a corresponding DNS request from the victim.

Network Traffic Analysis

dns The output contains both the DNS requests and corresponding responses.

`dns.flags.response == 1 && ip.src == 8.8.8.8` (isolates incoming DNS response packets specifically sent by Google's public DNS server (`8.8.8.8`).)

`dns.flags.response==1` (**DNS response packets** sent back by a server to answer a client's lookup request.)

`dns && dns.qry.name == "corp-login.acme-corp.local"` (DNS request for our Domain)

`dns.flags.response == 1 && ip.src == 8.8.8.8 && dns.qry.name == "corp-login.acme-corp.local"`
(internal DNS leak or misconfiguration by hunting for public DNS replies from Google (`8.8.8.8`) for an unroutable, private enterprise domain.)

`dns.flags.response == 1 && ip.src != 8.8.8.8 && dns.qry.name == "corp-login.acme-corp.local"`
(DNS responses other than the DNS Server.)

Answer the questions below

How many DNS responses were observed for the domain corp-login.acme-corp.local? 211

`dns.flags.response == 1 && dns.qry.name == "corp-login.acme-corp.local"`

The image shows a Wireshark packet capture interface. At the top, a filter is applied: `dns.flags.response == 1 && dns.qry.name == "corp-login.acme-corp.local"`. Below the filter, a list of packets is displayed, all of which are DNS responses from source IP 8.8.8.8 to various destination IPs. Packet 251 is selected, showing a destination of 192.168.10.18.

No.	Time	Source	Destination	Protocol	Length	Info
154	108523.271985	8.8.8.8	192.168.10.17	DNS	102	Standard
177	126972.409370	8.8.8.8	192.168.10.54	DNS	102	Standard
194	140262.081023	8.8.8.8	192.168.10.51	DNS	102	Standard
213	160636.981589	8.8.8.8	192.168.10.16	DNS	102	Standard
215	161324.272039	8.8.8.8	192.168.10.14	DNS	102	Standard
244	187512.600828	8.8.8.8	192.168.10.54	DNS	102	Standard
251	194373.426963	8.8.8.8	192.168.10.18	DNS	102	Standard
264	202651.610137	8.8.8.8	192.168.10.10	DNS	102	Standard
268	204547.072079	8.8.8.8	192.168.10.16	DNS	102	Standard
275	206672.450871	8.8.8.8	192.168.10.53	DNS	102	Standard
316	239677.818666	8.8.8.8	192.168.10.13	DNS	102	Standard
345	260566.361371	8.8.8.8	192.168.10.18	DNS	102	Standard

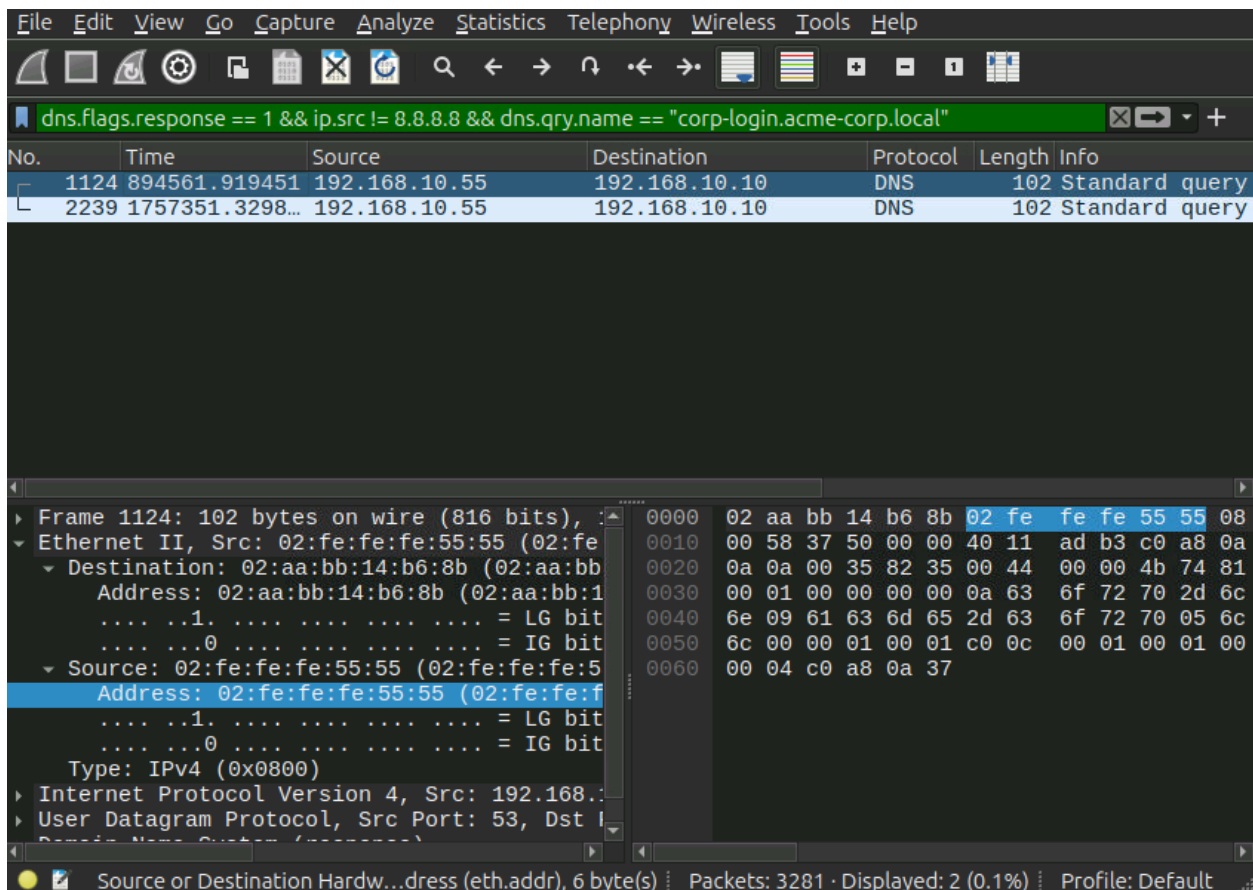
The detailed view of frame 251 shows the following structure:

- Frame 251: 102 bytes on wire (816 bits), 100 bytes captured (800 bits) on interface 0
- Ethernet II, Src: MS-NLB-PhysServer-08_08:08:08, Dst: 192.168.10.18
 - Destination: 02:aa:bb:78:09:55 (02:aa:bb:78:09:55)
 - Address: 02:aa:bb:78:09:55 (02:aa:bb:78:09:55)
 -1. = LG bit
 -0. = IG bit
 - Source: MS-NLB-PhysServer-08_08:08:08:08
 - Address: MS-NLB-PhysServer-08_08:08:08:08
 -1. = LG bit
 -0. = IG bit
 - Type: IPv4 (0x0800)
- Internet Protocol Version 4, Src: 8.8.8.8, Dst: 192.168.10.18
- User Datagram Protocol, Src Port: 53, Dst Port: 53
- DNS Standard query response (standard query response)

The packet bytes pane shows the raw data in hexadecimal and ASCII. The status bar at the bottom indicates: Name: Character string, Packets: 3281 · Displayed: 211 (6.4%) · Profile: Default.

How many DNS requests were observed from the IPs other than 8.8.8.8? 2

`dns.flags.response == 1 && ip.src != 8.8.8.8 && dns.qry.name == "corp-login.acme-corp.local"`



What IP did the attacker's forged DNS response return for the domain?
192.168.10.55

Task 6 - Spotting SSL Stripping in Action

SSL stripping is a man-in-the-middle technique in which an attacker intercepts and modifies traffic to remove or prevent TLS encryption between a client and a server. This causes the client to communicate over HTTP instead of HTTPS. The attacker retains a secure (HTTPS) session with the server while relaying plain HTTP to the victim, enabling eavesdropping and credential capture.

How It Works

1. **The victim initiates an HTTPS request** to a website.
2. **The attacker intercepts the request** using ARP spoofing or a rogue access point.

3. **The attacker connects to the website over HTTPS** but relays the response to the victim through HTTP.
4. **The victim unknowingly interacts over HTTP**, exposing sensitive data in plaintext.

Indicators of SSL stripping

- **Initial Request vs. Response:** The user's initial request may be for **HTTPS** (port 443), but the subsequent packets immediately shift to unencrypted **HTTP** (port 80) for the same domain.
- **Redirects/Link Rewriting:** Monitoring for redirects (HTTP Status Codes 301, 302) that persistently direct an HTTPS request to an HTTP resource.
- **Certificate Errors:** Although the attacker usually tries to hide this, the initial **TLS/SSL Handshake** may fail or display a self-signed certificate if the attacker uses a more direct proxying technique.

Network Traffic Analysis

`tls || ssl` (Narrowing down SSL traffic)

No.	Time	Source	Destination	Protocol	Length	Info
36	2025-09-07 14:00:56.217144	192.168.10.13	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
37	2025-09-07 14:00:56.517143	93.184.216.34	192.168.10.13	TLSv1.2	103	Server Hello
163	2025-09-08 14:06:54.365603	192.168.10.17	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
164	2025-09-08 14:06:54.665603	93.184.216.34	192.168.10.17	TLSv1.2	103	Server Hello
270	2025-09-09 14:04:25.867784	192.168.10.16	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
271	2025-09-09 14:04:26.167784	93.184.216.34	192.168.10.16	TLSv1.2	103	Server Hello
1854	2025-09-24 12:34:13.478528	192.168.10.13	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1855	2025-09-24 12:34:13.778528	142.250.72.14	192.168.10.13	TLSv1.2	103	Server Hello
1857	2025-09-24 12:47:24.462122	192.168.10.18	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1858	2025-09-24 12:47:24.762122	142.250.72.14	192.168.10.18	TLSv1.2	103	Server Hello
1859	2025-09-24 12:59:09.792634	192.168.10.15	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1860	2025-09-24 12:59:10.092634	93.184.216.34	192.168.10.15	TLSv1.2	103	Server Hello
1861	2025-09-24 13:00:50.888619	192.168.10.14	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1862	2025-09-24 13:00:51.188619	93.184.216.34	192.168.10.14	TLSv1.2	103	Server Hello
1865	2025-09-24 13:29:09.860985	192.168.10.16	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1866	2025-09-24 13:29:10.160985	142.250.72.14	192.168.10.16	TLSv1.2	103	Server Hello
1869	2025-09-24 13:39:26.956368	192.168.10.10	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1870	2025-09-24 13:39:27.256368	142.250.72.14	192.168.10.10	TLSv1.2	103	Server Hello
1871	2025-09-24 14:06:56.756009	192.168.10.15	93.184.216.34	TLSv1.2	148	[TCP Previous segment not captured] , Client Hello (SNI=corp-login.acme-corp.local)
1872	2025-09-24 14:06:57.056009	93.184.216.34	192.168.10.15	TLSv1.2	103	[TCP Previous segment not captured] , Server Hello
1873	2025-09-24 14:36:25.798014	192.168.10.12	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1874	2025-09-24 14:36:26.098014	93.184.216.34	192.168.10.12	TLSv1.2	103	Server Hello

`tls.handshake.type == 1 && tls.handshake.extensions_server_name == "corp-login.acme-corp.local"` (output is the confirmation that the domain in question uses TLS for communication. Examine the SSL traffic of our Server.)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
tls.handshake.type == 1 && tls.handshake.extensions_server_name == "corp-login.acme-corp.local"						
No.	Time	Source	Destination	Protocol	Length	Info
36	2025-09-07 14:00:56.217144	192.168.10.13	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
163	2025-09-08 14:06:54.365603	192.168.10.17	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
270	2025-09-09 14:04:25.867784	192.168.10.16	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1854	2025-09-24 12:34:13.478528	192.168.10.13	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1857	2025-09-24 12:47:24.462122	192.168.10.18	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1859	2025-09-24 12:59:09.792634	192.168.10.15	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1861	2025-09-24 13:00:50.888619	192.168.10.14	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1865	2025-09-24 13:29:09.860985	192.168.10.16	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1869	2025-09-24 13:39:26.956368	192.168.10.10	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1871	2025-09-24 14:06:56.756009	192.168.10.15	93.184.216.34	TLSv1.2	148	[TCP Previous segment not captured], Client Hello (SNI=corp-login.acme-corp.local)
1873	2025-09-24 14:36:25.798014	192.168.10.12	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1877	2025-09-24 14:52:51.294088	192.168.10.17	93.184.216.34	TLSv1.2	148	[TCP Previous segment not captured], Client Hello (SNI=corp-login.acme-corp.local)
1976	2025-09-25 12:23:22.820929	192.168.10.15	93.184.216.34	TLSv1.2	148	[TCP Previous segment not captured], Client Hello (SNI=corp-login.acme-corp.local)
1978	2025-09-25 12:28:15.252275	192.168.10.12	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1988	2025-09-25 13:32:16.428628	192.168.10.11	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1990	2025-09-25 13:36:08.443873	192.168.10.15	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
1992	2025-09-25 13:47:50.058069	192.168.10.15	93.184.216.34	TLSv1.2	148	[TCP Previous segment not captured], Client Hello (SNI=corp-login.acme-corp.local)
2003	2025-09-25 14:40:02.992097	192.168.10.15	142.250.72.14	TLSv1.2	148	[TCP Previous segment not captured], Client Hello (SNI=corp-login.acme-corp.local)
2096	2025-09-26 12:07:34.693297	192.168.10.11	93.184.216.34	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)
2099	2025-09-26 12:16:48.327670	192.168.10.10	142.250.72.14	TLSv1.2	148	Client Hello (SNI=corp-login.acme-corp.local)

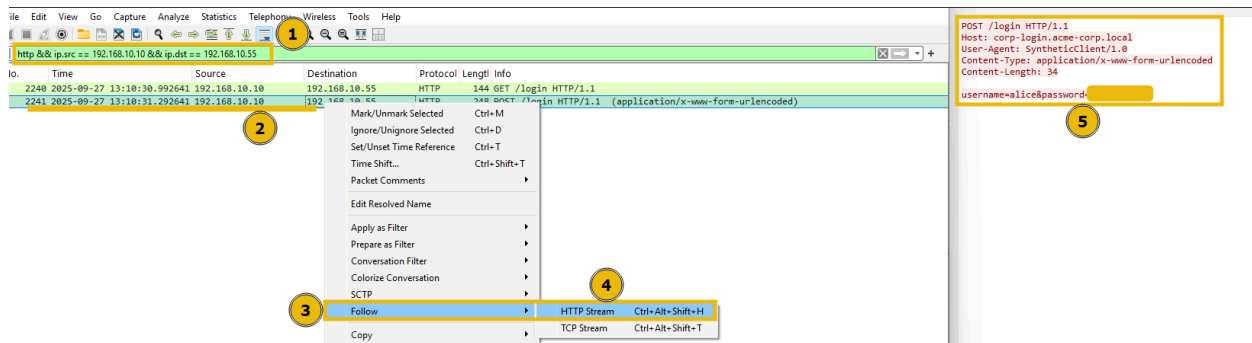
`dns.flags.response == 1 && ip.src == 192.168.10.55 && dns.qry.name == "corp-login.acme-corp.local"` (The above result shows two incidents in which the attacker's IP sent spoofed DNS responses for the domain of interest.)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
dns.flags.response == 1 && ip.src == 192.168.10.55 && dns.qry.name == "corp-login.acme-corp.local"						
No.	Time	Source	Destination	Protocol	Length	Info
1124	2025-09-17 13:30:41.102254	192.168.10.55	192.168.10.10	DNS	102	Standard query response 0x4b74 A corp-login.acme-corp.local A 192.168.10.55
2239	2025-09-17 13:30:51.102254	192.168.10.55	192.168.10.10	DNS	102	Standard query response 0x7689 A corp-login.acme-corp.local A 192.168.10.55

`http && ip.src == 192.168.10.10 && ip.dst == 192.168.10.55` (Verify TLS disappears. One of the main indicators of SSL stripping is that, after the spoof, the domain no longer performs TLS handshakes to the legitimate server, which validates the absence of TLS traffic from the victim to the real server.)

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help						
http && ip.src == 192.168.10.10 && ip.dst == 192.168.10.55						
No.	Time	Source	Destination	Protocol	Length	Info
2240	2025-09-27 13:10:30.992641	192.168.10.10	192.168.10.55	HTTP	144	GET /login HTTP/1.1
2241	2025-09-27 13:10:31.292641	192.168.10.10	192.168.10.55	HTTP	248	POST /login HTTP/1.1 (application/x-www-form-urlencoded)

We can clearly see that the victim connected to the server after SSL stripping and logged in.



Summary

- **ARP Spoofing (cache poisoning):**
 - **ARP spoof begins**, attacker sends unsolicited ARP **is-at** claiming gateway IP.
- **DNS Spoofing (forged DNS responses):**
 - **Victim DNS query** for **corp-login.acme-corp.local**.
 - **Forged DNS response** from attacker, answers domain → **192.168.10.55**.
- **SSL Stripping (TLS downgrade / credential capture):**
 - **The Victim initiates connection** to resolved IP, HTTP to attacker (SSL stripping effect).
 - **Credential POST observed** (cleartext).

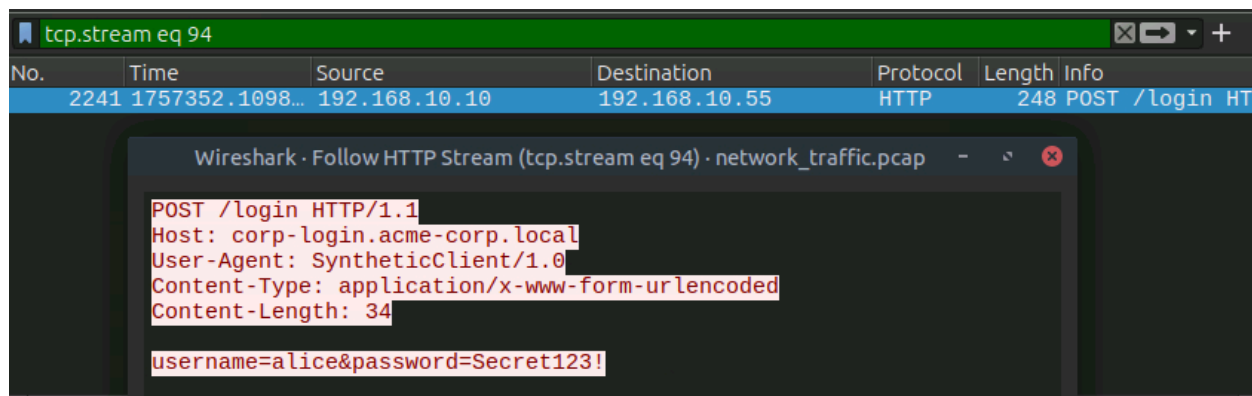
Answer the questions below

How many POST requests were observed for our domain corp-login.acme-corp.local? 1

run the query: tcp.stream eq 94 (isolates **the entire packet sequence of a single TCP connection session (Index #94)** from its opening handshake to its closure.)

What's the password of the victim found in the plaintext after successful ssl stripping attack.

Secret123!



Task 7 - Conclusion & Room Wrap-up

Some of the key concepts covered in this room are:

- Detected by finding **duplicate MAC addresses** for different IPs in the ARP cache (`arp -a`) or by spotting in Wireshark.
- **DNS Spoofing:** Unmasked by finding **multiple, conflicting DNS responses**.
- **SSL Stripping:** Exposed by finding sensitive data, like passwords, being sent in **plaintext over HTTP** to websites that should be secure.